TP — Conversions entre bases : décimal, binaire, hexadécimal

Objectifs

- connaître le vocabulaire : bit, quartet, octet, poids faible, poids fort ;
- convertir un nombre entre les bases 10, 2 et 16, dans les deux sens ;
- appliquer les conversions à des cas réels : masque et adresse réseau, adresse MAC, registre d'état, couleur, trame, codes ASCII, mesure d'un capteur.

Partie 1 — Le système positionnel

Vocabulaire

- **bit** : chiffre binaire, 0 ou 1 ;
- **quartet** (*nibble*) : groupe de 4 bits, soit exactement un chiffre hexadécimal ;
- **octet** (*byte*) : groupe de 8 bits, soit deux chiffres hexadécimaux ;
- **LSB** (*Least Significant Bit*) : bit de poids faible, le plus à droite. Dans un octet, c'est le bit 0 ;
- **MSB** (*Most Significant Bit*) : bit de poids fort, le plus à gauche. Dans un octet, c'est le bit 7.

```
Numéro du bit :   7   6   5   4   3   2   1   0
                MSB                         LSB
```

Notations

Sans indication de base, `10` peut valoir dix, deux ou seize : il faut **toujours préciser la base**.

Base	Notation mathématique	Notation en C, C++, Python	Exemple
10 (décimal)	156_{10}	`156`	156
2 (binaire)	10011100_{2}	`0b10011100`	10011100
16 (hexadécimal)	$9C_{16}$	`0x9C`	9C

> Le préfixe `0b` est standard en Python, en C++ depuis C++14 et en C depuis C23 ; GCC l'accepte aussi en C plus ancien comme extension. Le préfixe `0x` est accepté partout.

Principe

Dans une base b, chaque chiffre est multiplié par une **puissance de b** qui dépend de sa **position**, numérotée à partir de 0 en partant de la droite.

```
Base 10 : 156        = 1 x 10^2 + 5 x 10^1 + 6 x 10^0
Base 2  : 10011100   = 1 x 2^7 + 0 x 2^6 + 0 x 2^5 + 1 x 2^4 + 1 x 2^3 + 1 x 2^2 + 0 x 2^1 + 0 x
2^0
Base 16 : 9C         = 9 x 16^1 + 12 x 16^0
```

Tables à connaître par cœur :

Décimal	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
Hexadécimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Binaire	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Puissance de 2	2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}	2^{12}	2^{16}
Valeur	1	2	4	8	16	32	64	128	256	512	1 024	4 096	65 536

Puissance de 16	16^0	16^1	16^2	16^3	16^4
Valeur	1	16	256	4 096	65 536

Repères :

- avec n bits, on représente 2^n valeurs, de 0 à $2^n - 1$: un octet (8 bits) va de 0 à 255, soit de `0x00` à `0xFF` ;
- un chiffre hexadécimal vaut exactement 4 bits : un octet s'écrit avec 2 chiffres hexadécimaux, un entier de 16 bits avec 4, un entier de 32 bits avec 8 ;
- $16^n = 2^{4n}$: c'est pour cela que la conversion entre les bases 2 et 16 se fait sans calcul (partie 5).

Partie 2 — Binaire → décimal

Méthode 1 : somme des poids

On additionne les poids des bits à 1.

```
Bits     :   1   0   0   1   1   1   0   0
Poids    : 128  64  32  16   8   4   2   1
Retenus  : 128          16   8   4         -> 128 + 16 + 8 + 4 = 156
```

Méthode 2 : de gauche à droite (méthode de Horner)

On part de 0. Pour chaque bit, de gauche à droite : **valeur = valeur × 2 + bit**. Cette méthode n'exige pas de connaître les puissances ; c'est aussi celle qu'utilisent les programmes de conversion.

```
bit -> valeur × 2 + bit = valeur

1 ->  0 × 2 + 1 = 1
0 ->  1 × 2 + 0 = 2
0 ->  2 × 2 + 0 = 4
1 ->  4 × 2 + 1 = 9
1 ->  9 × 2 + 1 = 19
```

```
1 -> 19 × 2 + 1 = 39
0 -> 39 × 2 + 0 = 78
0 -> 78 × 2 + 0 = 156
```

Exercice 1 — Convertissez en décimal, avec la méthode de votre choix :

Binaire	`1011`	`11111111`	`10000000`	`1100100`	`00101010`	`1010101010`
Décimal						

Partie 3 — Décimal → binaire

Méthode 1 : divisions successives par 2

On divise par 2 jusqu'à obtenir un quotient nul. Les **restes**, lus **de bas en haut**, donnent les bits, du poids fort au poids faible.

```
156 / 2 = 78  reste 0   <- bit 0 (poids faible)
 78 / 2 = 39  reste 0
 39 / 2 = 19  reste 1
 19 / 2 =  9  reste 1
  9 / 2 =  4  reste 1
  4 / 2 =  2  reste 0
  2 / 2 =  1  reste 0
  1 / 2 =  0  reste 1   <- bit 7 (poids fort)

Lecture de bas en haut : 10011100
```

Le même calcul, en présentation « en escalier » :

```
156 | 2
  0 | 78 | 2
       0 | 39 | 2
            1 | 19 | 2
                 1 |  9 | 2
                      1 |  4 | 2
                           0 |  2 | 2
                                0 |  1 | 2
                                     1 |  0
```

En lisant les restes de bas en haut : $156_{10} = 10011100_{2}$.

Méthode 2 : soustraction des puissances de 2

On retire la plus grande puissance de 2 possible, et on recommence : 156 − 128 = 28 ; 28 − 16 = 12 ; 12 − 8 = 4 ; 4 − 4 = 0. Les bits 7, 4, 3 et 2 sont à 1, les autres à 0 : `10011100`.

Exercice 2 — Convertissez en binaire. Écrivez le résultat sur 8 bits quand c'est possible.

Décimal	**13**	**64**	**100**	**200**	**255**	**1000**
Binaire						

Combien de bits faut-il au minimum pour écrire 1000 ? Justifiez avec les puissances de 2.

Partie 4 — Hexadécimal ⟷ décimal

Hexadécimal → décimal

Chaque chiffre est multiplié par une puissance de 16 (1, 16, 256, 4 096…).

```
0x3E8 = 3 x 256 + 14 x 16 + 8 x 1 = 768 + 224 + 8 = 1000
```

La méthode de Horner fonctionne aussi : **valeur = valeur × 16 + chiffre**.

```
3 ->   0 × 16 + 3  = 3
E ->   3 × 16 + 14 = 62
8 ->  62 × 16 + 8  = 1000
```

Décimal → hexadécimal : divisions successives par 16

Même principe qu'en base 2, mais on divise par 16. Chaque reste (de 0 à 15) est remplacé par son chiffre hexadécimal.

```
1000 / 16 = 62  reste 8    -> 8
  62 / 16 =  3  reste 14   -> E
   3 / 16 =  0  reste 3    -> 3

Lecture de bas en haut : 3E8
```

Exercice 3 —

Hexadécimal	`0x1A`	`0x7F`	`0xC0`	`0x2BC`	`0xFFFF`
Décimal					

Décimal	**75**	**160**	**254**	**4096**	**49152**
Hexadécimal					

Partie 5 — Hexadécimal ⟷ binaire

C'est la conversion la plus simple : **un chiffre hexadécimal = un quartet (4 bits)**. Il n'y a aucun calcul, seulement la table de la partie 1.

Hexadécimal → binaire : on remplace chaque chiffre par ses 4 bits, sans oublier les zéros de tête.

```
0x9C  ->  9    C
          1001 1100   ->  10011100
```

Binaire → hexadécimal : on forme des groupes de 4 bits **en partant de la droite**, on complète le groupe de gauche avec des 0, puis on remplace chaque groupe par son chiffre.

```
101111  ->  10 1111  ->  0010 1111  ->  2F
```

> **Erreur classique :** grouper à partir de la gauche. `101111` découpé en `1011 11`, puis complété à droite en `1011 1100`, donne `BC` (188) au lieu de `2F` (47). Des zéros ajoutés **à gauche** ne changent pas la valeur ; ajoutés **à droite**, ils la multiplient.

Exercice 4 —

Hexadécimal	`0xA5`	`0x3C`	`0x7E`	`0x1F4`	`0xBEEF`
Binaire					

Binaire	`11010110`	`1110011`	`1000000001`	`11111111111`
Hexadécimal				

Récapitulatif des méthodes

Conversion	Méthode
2 → 10	somme des poids, ou Horner (× 2)
10 → 2	divisions successives par 2, ou soustraction des puissances de 2
16 → 10	somme des poids (puissances de 16), ou Horner (× 16)
10 → 16	divisions successives par 16
16 ? 2	remplacement chiffre par chiffre (quartets)

> **Astuce :** pour convertir un grand nombre décimal en binaire, passez par l'hexadécimal. 1000 → `0x3E8` → `0011 1110 1000` : trois divisions par 16 au lieu de dix divisions par 2.

Point de contrôle : faites vérifier vos réponses des exercices 1 à 4 par l'enseignant, ou vérifiez-les vous-même avec la calculatrice en mode *Programmeur*, ou en Python :

```python
int("10011100", 2)   # 156
int("3E8", 16)       # 1000
bin(156)             # '0b10011100'
hex(1000)            # '0x3e8'
```

Partie 6 — Applications

Exercice 5 — Masque de sous-réseau. Écrivez le masque `/26` (`255.255.255.192`) en binaire (4 octets séparés par des points), puis en hexadécimal sur 32 bits. Combien y a-t-il de bits à 1 ? Combien de bits restent pour identifier les hôtes ? Combien d'adresses le sous-réseau contient-il, et combien d'hôtes peut-on y adresser ?

Exercice 6 — Registre d'état. Un capteur renvoie l'octet d'état `0x5A`. Les bits 0 à 7 signifient, dans l'ordre : prêt, mesure valide, alarme haute, alarme basse, calibré, défaut, mode veille, erreur de communication. Convertissez `0x5A` en binaire et listez les indicateurs actifs.

> Le bit 0 est le bit de poids faible (LSB), situé à droite de l'écriture binaire.

Exercice 7 — Couleur. Une couleur web s'écrit `#RRGGBB`, chaque composante sur un octet en hexadécimal. Donnez les valeurs décimales du rouge, du vert et du bleu de `#FF8800`. Écrivez la couleur (R = 32, G = 178, B = 170) au format `#RRGGBB`.

Exercice 8 — Trame. Un analyseur relève les octets `0x4F 0x4B 0x0D 0x0A`. Convertissez-les en décimal, puis identifiez les caractères avec la table ASCII de l'annexe A. Les deux derniers octets sont des caractères de contrôle : lesquels ?

Exercice 9 — Lire une suite d'octets. Une trame contient les octets hexadécimaux `02 10 2D C8 FF`. Écrivez pour chacun la valeur décimale et les huit bits, sans inverser l'ordre des octets.

Partie 7 — Applications complémentaires

Exercice 10 — Adresse MAC. Une adresse MAC compte 6 octets. Dans le **premier** octet, deux bits ont un rôle particulier :

- bit 0 (I/G) : 0 = adresse individuelle (*unicast*), 1 = adresse de groupe (*multicast* ou diffusion) ;
- bit 1 (U/L) : 0 = adresse universelle, attribuée par le constructeur ; 1 = adresse administrée localement.

Pour chaque adresse, écrivez le premier octet en binaire et déduisez son type.

Adresse MAC	1er octet en binaire	Unicast / groupe	Universelle / locale
`00:1A:2B:3C:4D:5E`			
`01:00:5E:00:00:FB`			
`02:42:AC:11:00:02`			
`FF:FF:FF:FF:FF:FF`			

Bonus : la troisième adresse est typique d'un conteneur Docker. Convertissez ses quatre derniers octets en décimal : que reconnaissez-vous ?

Exercice 11 — Adresse réseau. Un poste a l'adresse `192.168.10.77/26`. L'adresse du réseau s'obtient par un **ET logique** bit à bit entre l'adresse et le masque (1 ET 1 = 1, sinon 0).

1. Écrivez l'adresse IP en binaire (4 octets), puis en hexadécimal sur 32 bits.
2. Effectuez le ET logique avec le masque de l'exercice 5 et donnez l'adresse du réseau en notation décimale pointée.
3. L'adresse de diffusion s'obtient en mettant à 1 tous les bits de la partie hôte. Donnez-la.
4. Donnez la plage des adresses utilisables par les hôtes.

Exercice 12 — Convertisseur analogique-numérique. Un microcontrôleur mesure une tension avec un CAN de 10 bits sous une référence de 3,3 V. La tension se calcule par U = N × 3,3 / 1023, où N est la valeur lue dans le registre.

1. Quelle est la plus grande valeur N possible, en décimal et en hexadécimal ?
2. Le registre renvoie `0x2E6`. Calculez N en décimal, puis la tension mesurée (deux décimales).
3. Combien de chiffres hexadécimaux faut-il pour afficher une valeur de 10 bits ? Combien d'octets pour la stocker en mémoire ?

Exercice 13 — Masques en C. On reprend le registre d'état de l'exercice 6. Chaque question part de `etat = 0x5A`.

```c
uint8_t etat = 0x5A;

if (etat & 0x08) { /* ... */ }   /* question 1 */
etat = etat | 0x01;              /* question 2 */
etat = etat & ~0x40;             /* question 3 (sur 8 bits, ~0x40 vaut 0xBF) */
```

1. Quel bit le masque `0x08` teste-t-il ? Quel est le résultat de `etat & 0x08` ? La condition est-elle vraie ? Quel indicateur est ainsi testé ?
2. Que vaut `etat | 0x01` en hexadécimal ? Quel indicateur a été modifié, et comment ?
3. Que vaut `etat & ~0x40` en hexadécimal ? Quel indicateur a été modifié, et comment ?
4. Écrivez en hexadécimal le masque qui permet de tester l'indicateur « défaut ».

Exercice 14 — Codes ASCII. Utilisez la table de l'annexe A.

1. Décodez la suite d'octets `42 54 53 20 43 49 45 4C`.
2. Codez en hexadécimal la chaîne `IR-2`.
3. Écrivez les codes de `A` et de `a` en binaire. Quel bit les distingue ? Déduisez-en une méthode pour passer d'une majuscule à la minuscule correspondante.
4. Le caractère `7` a pour code `0x37`. Comment obtenir la valeur numérique 7 à partir de ce code ?

> Un chiffre reçu sur une liaison série ou lu au clavier est un **caractère**, pas un nombre : `'7'` vaut 55, pas 7.

Exercice 15 — Somme de contrôle. Un protocole série ajoute en fin de trame un octet de contrôle : la somme de tous les octets de données, dont on ne garde que les 8 bits de poids faible. Les données sont `12 34 A0 7F`.

1. Convertissez les octets en décimal et calculez leur somme.
2. Écrivez cette somme en hexadécimal. Pourquoi ne tient-elle pas sur un octet ?
3. Quelle est la valeur de l'octet de contrôle, en hexadécimal ?

Exercice 16 — Ports dans une capture réseau. Dans un en-tête TCP ou UDP, un numéro de port occupe 2 octets, transmis poids fort en premier (ordre réseau, dit *big-endian*). Une capture Wireshark affiche les champs de port suivants. Convertissez-les en décimal et identifiez le service.

Octets	`01 BB`	`00 50`	`00 35`	`1F 90`
Port (décimal)				
Service				

Quelle est la plus grande valeur possible d'un numéro de port ? Justifiez.

Annexe A — Table ASCII (0 à 127)

L'ASCII code chaque caractère sur **7 bits** (0 à 127, soit `0x00` à `0x7F`). Dans la table, la **ligne** donne le chiffre hexadécimal de poids fort et la **colonne** le chiffre de poids faible.

Exemple : `A` est à la ligne `4_` et à la colonne `_1`, donc son code est `0x41` = 4 × 16 + 1 = 65.

	_0	**_1**	**_2**	**_3**	**_4**	**_5**	**_6**	**_7**	**_8**	**_9**	**_A**	**_B**	**_C**	**_D**	**_E**	**_F**
0_	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1_	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2_	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Repères utiles :

Caractère	**Hexadécimal**	**Décimal**	**Remarque**
espace (SP)	`0x20`	32	premier caractère imprimable
`0` à `9`	`0x30` à `0x39`	48 à 57	valeur du chiffre = code − `0x30`
`A` à `Z`	`0x41` à `0x5A`	65 à 90	
`a` à `z`	`0x61` à `0x7A`	97 à 122	minuscule = majuscule + `0x20`

Caractères de contrôle (non imprimables) :

Déc.	**Hex.**	**Sigle**	**Signification**
0	`00`	NUL	caractère nul, fin de chaîne en C (`\0`)
1	`01`	SOH	début d'en-tête
2	`02`	STX	début de texte
3	`03`	ETX	fin de texte
4	`04`	EOT	fin de transmission

Déc.	Hex.	Sigle	Signification
5	`05`	ENQ	demande
6	`06`	ACK	accusé de réception positif
7	`07`	BEL	sonnerie (`\a`)
8	`08`	BS	retour arrière (`\b`)
9	`09`	HT	tabulation horizontale (`\t`)
10	`0A`	LF	saut de ligne (`\n`)
11	`0B`	VT	tabulation verticale (`\v`)
12	`0C`	FF	saut de page (`\f`)
13	`0D`	CR	retour chariot (`\r`)
14	`0E`	SO	hors code
15	`0F`	SI	en code
16	`10`	DLE	échappement de transmission
17	`11`	DC1	commande d'appareil 1 (XON)
18	`12`	DC2	commande d'appareil 2
19	`13`	DC3	commande d'appareil 3 (XOFF)
20	`14`	DC4	commande d'appareil 4
21	`15`	NAK	accusé de réception négatif
22	`16`	SYN	synchronisation
23	`17`	ETB	fin de bloc de transmission
24	`18`	CAN	annulation
25	`19`	EM	fin de support
26	`1A`	SUB	substitution
27	`1B`	ESC	échappement (`\033`)
28	`1C`	FS	séparateur de fichiers
29	`1D`	GS	séparateur de groupes
30	`1E`	RS	séparateur d'enregistrements
31	`1F`	US	séparateur d'unités
127	`7F`	DEL	suppression

> STX, ETX, ACK et NAK servent encore dans de nombreux protocoles série (lecteurs de badges, balances, automates). XON et XOFF assurent le contrôle de flux logiciel d'une liaison RS-232.

Annexe B — Table ASCII étendue (128 à 255)

L'ASCII n'utilise que 7 bits. Le 8e bit d'un octet permet 128 codes supplémentaires (`0x80` à `0xFF`), mais **leur contenu dépend de la table de codage choisie** : il n'existe pas un seul « ASCII étendu ». La table ci-dessous est **Windows-1252**, la plus répandue en Europe de l'Ouest. Elle est identique à **ISO 8859-1 (Latin-1)** de `0xA0` à `0xFF` ; de `0x80` à `0x9F`, ISO 8859-1 place des caractères de contrôle à la place des symboles affichés ici.

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
8_	€	—	‚	ƒ	„	…	†	‡	ˆ	‰	Š	‹	Œ	—	Ž	—
9_	—	‘	’	“	”	•	–	—	˜	™	š	›	œ	—	ž	Ÿ
A_	NBSP	¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	SHY	®	¯
B_	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C_	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D_	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E_	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F_	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

— : code non attribué. NBSP (`0xA0`) : espace insécable. SHY (`0xAD`) : trait d'union conditionnel.

> **Attention à l'UTF-8.** Aujourd'hui, la plupart des fichiers et des pages web sont codés en **UTF-8**, où les 128 premiers codes sont ceux de l'ASCII mais où un caractère accentué occupe **plusieurs octets** : `é` vaut `0xE9` en Windows-1252, mais `0xC3 0xA9` en UTF-8. Si un texte UTF-8 est lu comme du Windows-1252, ces deux octets s'affichent `Ã©` (voir la table) : c'est l'origine des accents « cassés ».

Annexe C — Éléments de correction

Exercices 1 à 4

Exercice	Réponses
1	11 ; 255 ; 128 ; 100 ; 42 ; 682
2	0000 1101 ; 0100 0000 ; 0110 0100 ; 1100 1000 ; 1111 1111 ; 11 1110 1000. Pour 1000 : $2^9 = 512 \leq 1000 < 1\,024 = 2^{10}$, donc **10 bits**
3	26 ; 127 ; 192 ; 700 ; 65 535 — puis `4B` ; `A0` ; `FE` ; `1000` ; `C000`
4	1010 0101 ; 0011 1100 ; 0111 1110 ; 0001 1111 0100 ; 1011 1110 1110 1111 — puis `D6` ; `73` ; `201` ; `7FF`

Exercice 5

- Binaire : `11111111.11111111.11111111.11000000` ; hexadécimal : `0xFFFFFFC0`.
- 26 bits à 1, donc 32 − 26 = **6 bits** pour les hôtes.
- 2^6 = 64 adresses dans le sous-réseau, dont 62 utilisables par les hôtes (on retire l'adresse du réseau et l'adresse de diffusion).

Exercice 6

`0x5A` = `0101 1010` : les bits 1, 3, 4 et 6 sont à 1.

Bit	Valeur	Indicateur	État
0	0	Prêt	Inactif
1	1	Mesure valide	Actif
2	0	Alarme haute	Inactif
3	1	Alarme basse	Actif
4	1	Calibré	Actif
5	0	Défaut	Inactif
6	1	Mode veille	Actif
7	0	Erreur de communication	Inactif

Indicateurs actifs : mesure valide, alarme basse, calibré, mode veille.

Exercice 7

- `#FF8800` : rouge `FF` = 255, vert `88` = 136, bleu `00` = 0.
- (32, 178, 170) : 32 = `20`, 178 = `B2`, 170 = `AA`, d'où `#20B2AA`.

Exercice 8

Hexadécimal	Décimal	ASCII
`0x4F`	79	`O`
`0x4B`	75	`K`
`0x0D`	13	CR : retour chariot (*Carriage Return*, `\r`)
`0x0A`	10	LF : saut de ligne (*Line Feed*, `\n`)

La trame contient la chaîne `OK` suivie de la fin de ligne CR LF, très utilisée dans les protocoles texte (commandes AT, HTTP, SMTP).

Exercice 9

Hexadécimal	Décimal	Binaire
`02`	2	`00000010`
`10`	16	`00010000`
`2D`	45	`00101101`
`C8`	200	`11001000`
`FF`	255	`11111111`

Exercice 10

Adresse MAC	1er octet en binaire	Unicast / groupe	Universelle / locale
`00:1A:2B:3C:4D:5E`	`0000 0000`	unicast	universelle
`01:00:5E:00:00:FB`	`0000 0001`	groupe (multicast)	universelle
`02:42:AC:11:00:02`	`0000 0010`	unicast	administrée localement
`FF:FF:FF:FF:FF:FF`	`1111 1111`	groupe (diffusion)	— (adresse de diffusion)

Bonus : `AC 11 00 02` = 172.17.0.2, l'adresse IPv4 du conteneur dans le réseau Docker par défaut.

Exercice 11

1. `11000000.10101000.00001010.01001101` ; `0xC0A80A4D`.
2. Seul le dernier octet change : `01001101` ET `11000000` = `01000000` = 64. Réseau : **192.168.10.64**.
3. Partie hôte (6 bits) à 1 : `01111111` = 127. Diffusion : **192.168.10.127**.
4. De 192.168.10.65 à 192.168.10.126, soit 62 hôtes.

Exercice 12

1. $2^{10} - 1$ = **1023** = `0x3FF`.
2. `0x2E6` = 2 × 256 + 14 × 16 + 6 = **742** ; U = 742 × 3,3 / 1023 ≈ **2,39 V**.
3. 10 bits occupent 3 quartets, donc **3 chiffres** hexadécimaux ; en mémoire, il faut **2 octets** (6 bits inutilisés).

Exercice 13

1. `0x08` = `0000 1000` teste le bit 3. `0x5A & 0x08` = `0x08`, non nul : la condition est vraie, l'alarme basse est active.
2. `0x5A | 0x01` = `0x5B` : le bit 0 est mis à 1, l'indicateur « prêt » devient actif.
3. `0x5A & 0xBF` = `0x1A` : le bit 6 est mis à 0, le capteur sort du mode veille.
4. Bit 5 : `0x20` (en C, on écrit aussi `1 << 5`).

Exercice 14

1. `BTS CIEL` (`0x20` est l'espace).
2. `I R - 2` → `49 52 2D 32`.
3. `A` = `0100 0001`, `a` = `0110 0001` : seul le bit 5 diffère (valeur 32 = `0x20`). Minuscule = majuscule + `0x20`, ou mise à 1 du bit 5 (`c | 0x20`).
4. 7 = `0x37` − `0x30`. En C : `valeur = c - '0';`

Exercice 15

1. 18 + 52 + 160 + 127 = **357**.
2. 357 = `0x165` : elle dépasse 255 (`0xFF`), il faut 9 bits.
3. On garde les 8 bits de poids faible : `0x65` (101, soit 357 − 256).

Exercice 16

Octets	`01 BB`	`00 50`	`00 35`	`1F 90`
Port (décimal)	443	80	53	8080
Service	HTTPS	HTTP	DNS	HTTP alternatif (proxy, serveur de développement)

Un port occupe 16 bits : la plus grande valeur est $2^{16} - 1$ = **65 535** (`0xFFFF`).